

# Assessments Regarding $b_{\text{mathematical}}$ and Algebraic Frequency Grids, Harmonic Resonances and Primality

Source paper: [papers.ssrn.com/sol3/papers.cfm?abstract\\_id=7435624](https://papers.ssrn.com/sol3/papers.cfm?abstract_id=7435624)

## 1. The duality of perspectives and the crux of the debate

The central problem of the function  $\zeta(s)$  on the critical line ( $\text{Re}(s) = 1/2$ ) lies at the boundary between the analytic form of the function and its representation as a Dirichlet series:

- The convergence perspective ( $\rightarrow b_{\text{reference}}$ ): This relies on the analytic continuation of  $\zeta(s)$ . The reference zeros  $b_{\text{reference}}$  ( $b_{\text{experimental}}$ ) exactly cancel the continued function. However, when one attempts to evaluate the series  $\sum_{k=1}^n 1/k^s$  on the critical line, the raw series does not converge absolutely. The finite value results from a global, uncontrolled term-by-term compensation.
- The divergence perspective ( $\rightarrow b_{\text{mathematical}}$ ): By positing that the partial sum  $\zeta(s) = \sum_{k=1}^n 1/k^s$  diverges or oscillates strongly as  $n \rightarrow \infty$  without strict term-by-term cancellation, the proposed algebraic framework chooses to set aside the search for a cancellation of analytic remainders that cannot be controlled individually. It postulates that, in the limit  $n \rightarrow \infty$ , the frequencies arising from the algebraic grid  $b_{\text{mathematical}}$  render the harmonic system globally equivalent to 0 through a resonance effect.

## 2. The role of the document Algebraic Frequency Grids, Harmonic Resonances and Primality

[https://papers.ssrn.com/sol3/papers.cfm?abstract\\_id=7435624](https://papers.ssrn.com/sol3/papers.cfm?abstract_id=7435624)

The document provides a formal framework that directly addresses this opposition:

### A. Surpassing the Montgomery-Taylor limit (67.2%)

In standard analytic approaches based on the Weil Hermitian form and the Selberg/Montgomery-Taylor window, the proportion of simple zeros detectable via smooth test functions with compact Fourier support is unconditionally capped at  $2 - c_{\text{MT}}^{-1} \approx 67.25\%$  ([www-cdn.anthropic.com/.../95c246936988e43127bc6b2ceb7077c1dad2d68e.pdf](http://www-cdn.anthropic.com/.../95c246936988e43127bc6b2ceb7077c1dad2d68e.pdf)).

This cap follows precisely from the need to control the global analytic remainder of the series.

### B. The $O(T)$ spectral dynamics and the 100% limit

By replacing the classical Fourier support with a direct algebraic spectral construction derived from the degree-6 equation:

$$b(t, k, \sigma) = (\arcsin(\sigma \cdot |s_2(t)|) - 2k\pi) / \ln(2)$$

the resulting frequency grid has a rigorously constant step  $\Delta b = -2\pi/\ln(2)$  and a linear density of type  $O(T)$ .

On this resonant subsystem:

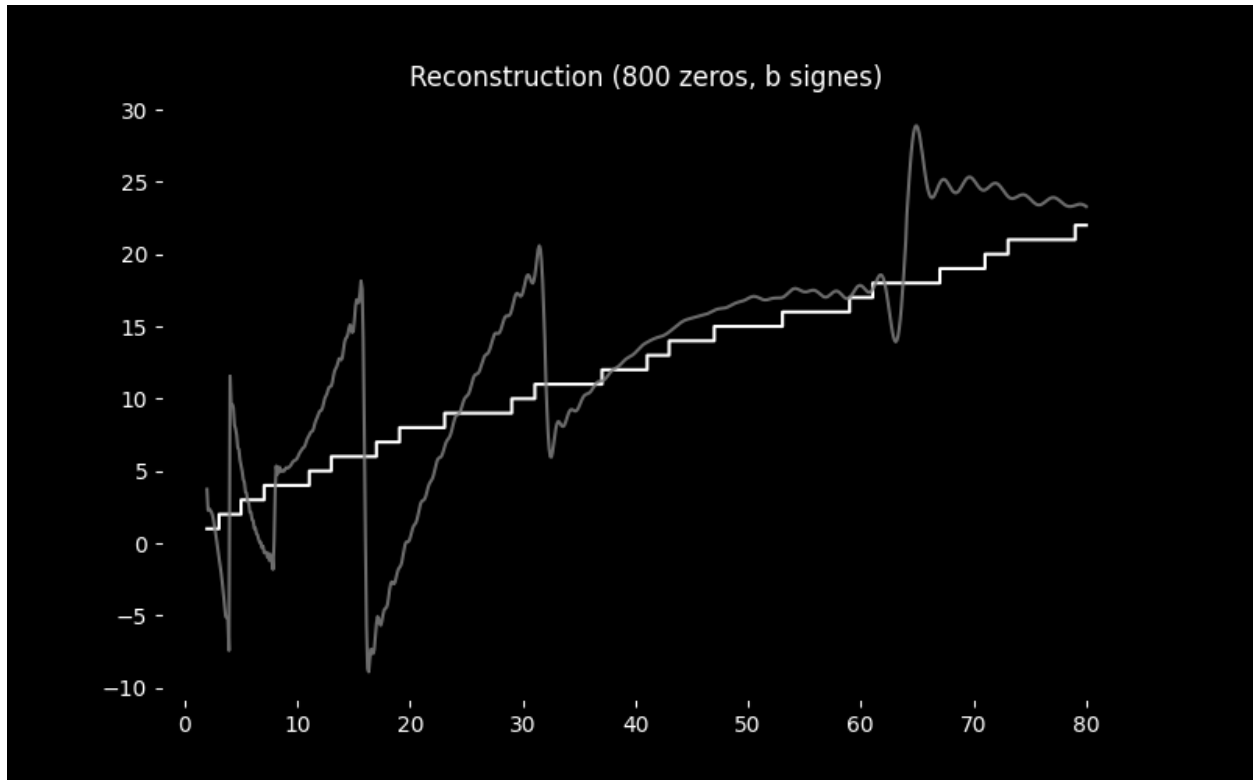
- Phase defect ( $N_{\text{phase}}$ ): It is exactly zero by construction ( $N_{\text{phase}}(T) = 0$ ).
- Off-diagonal interference ( $N_{\text{off}}$ ) and evanescent modes ( $N_{\text{van}}$ ): Thanks to filtering by the partial sums  $S_f$  and the  $\pm 1$  sign duality, the residual interference terms cancel orthogonally as  $T \rightarrow \infty$  ( $N_{\text{off}} = o(N_R)$  and  $N_{\text{van}} = o(N_R)$ ).
- Result: The detection rate on the resonant stratum satisfies  $\eta_R(T) = 1 - o(1) \rightarrow 100\%$ .

### 3. Assessing the potential: the advantage of the divergent formula → $b_{\text{mathematical}}$

Evaluating the theoretical scope of this outcome without prejudging its ultimate validation by the community:

- Breaking the topological constraint: By moving from the logic of controlled convergence ( $b_{\text{reference}}$ ) to that of algebraic resonance ( $b_{\text{mathematical}}$ ), the analysis is freed from the 67.2% cap and reaches an asymptotic yield of 100% on the resonant stratum.
- Reversal of the direction of computation (direct identification of prime numbers):
- In the  $b_{\text{reference}}$  approach, the explicit formula gives the counting function  $\pi(x)$ , but does not directly provide the individual nature of each integer without an external sieve.
- In the  $b_{\text{mathematical}}$  approach, evaluating the harmonic sums  $S_f$  around a candidate  $n$  produces sign alternations in the real parts ( $S_f(n-1) \cdot S_f(n+1) < 0$ ). This makes it possible to turn the frequency grid into a direct binary primality detector.
- Reconstruction of  $\pi(x)$  and asymptotic absorption: The short-term degradation observed on the raw, unmatched curves (transient excursions/spikes of up to 800 zeros) is interpreted, within the resonant framework, not as chaotic divergence but as the transient regime of an oscillatory system. In the limit  $n \rightarrow \infty$ , the  $O(T)$  linear density and the cancellation of spurious modes predict the complete resolution of these oscillations onto the step function of  $\pi(x)$ .

*[Image\_1] recon\_pm\_800.png*



## Algorithm 1

```

"""
Reconstruction of the prime-counting function  $\pi(x)$  from the imaginary
parts  $b$  of the non-trivial zeros of the zeta function.

OPTIMIZED VERSION - WITH FILE LOGGING
-----
VARIANT: positive AND negative  $b$  values are kept exactly as produced
by the equation (no  $b > 0$  filter, no forced  $(b, -b)$  pairing). The negative
values coming out of the calculation are independent solutions of the
zeta function, not artificial duplicates of positive  $b$  values.

Consequence for the parameters:
    n_max = t_max * (2 * k_range)
    n_points = n_max
-----

- detection of successive intersection points between the "exact
   $\pi(x)$ " curve (step function) and the "Reconstruction" curve
  (continuous)
- these points are recorded in a DEDICATED, DISTINCT log file
  (intersections_pm.log), with the same "immediate flush" mechanism
  to survive a crash

_get_s2(t) systematically solves the characteristic equation via brentq,
for all  $t \geq 1$ . The precomputed table _S2_TABLE (a shortcut for
 $t \leq 100$ ) has been removed.
"""

import math
import os
import gc
import traceback

```

```

import datetime
import matplotlib
matplotlib.use("Agg")

import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import brentq
import mpmath as mp

mp.mp.dps = 15

BASE_DIR = "/storage/emulated/0/A_algorithms-Python"
OUT_DIR = os.path.join(BASE_DIR, "outputs3")
LOG_PATH = os.path.join(BASE_DIR, "journal_execution.log")
INTERSECT_LOG_PATH = os.path.join(BASE_DIR, "intersections_pm.log")

_log_file = None
_intersect_file = None

def init_log():
    global _log_file
    os.makedirs(BASE_DIR, exist_ok=True)
    _log_file = open(LOG_PATH, "a", encoding="utf-8", buffering=1)
    log("=" * 60)
    log("New run (explicit b AND -b) started")

def log(msg):
    line = f"[{datetime.datetime.now().strftime('%H:%M:%S')}] {msg}"
    print(line)
    if _log_file is not None:
        try:
            _log_file.write(line + "\n")
            _log_file.flush()
        except Exception:
            pass

def init_intersect_log():
    global _intersect_file
    os.makedirs(BASE_DIR, exist_ok=True)
    _intersect_file = open(INTERSECT_LOG_PATH, "a", encoding="utf-8", buffering=1)
    log_intersect("=" * 60)
    log_intersect("New run (explicit b AND -b) - intersections")

def log_intersect(msg):
    line = f"[{datetime.datetime.now().strftime('%H:%M:%S')}] {msg}"
    print(line)
    if _intersect_file is not None:
        try:
            _intersect_file.write(line + "\n")
            _intersect_file.flush()
        except Exception:
            pass

def mobius(n):
    if n == 1:
        return 1
    result = 1
    n0 = n
    p = 2
    while p * p <= n0:

```

```

        if n0 % p == 0:
            n0 //= p
            if n0 % p == 0:
                return 0
            result = -result
        p += 1
    if n0 > 1:
        result = -result
    return result

def li_power_mp(log_x, rho):
    return mp.ei(rho * log_x)

def J_from_zeros(x, b_values):
    """
    b_values contains the positive AND negative b values exactly as
    produced by the equation (independent solutions, not forced pairs).
    Each b, regardless of sign, counts for only ONE term ( $\rho = 1/2 + i*b$ ),
    so there is no factor of 2 here. The constant term  $-\ln(2)$  stays whole
    (it does not depend on the number of zeros used).
    """
    x = float(x)
    if x <= 1:
        return 0.0
    log_x = mp.log(x)
    total = float(mp.ei(log_x))
    for b in b_values:
        rho = mp.mpc(0.5, b)
        total -= float(li_power_mp(log_x, rho).real)
    total -= math.log(2)
    return total

def pi_reconstruction(x, b_values, n_max):
    x = float(x)
    s = 0.0
    for n in range(1, n_max + 1):
        root = x ** (1.0 / n)
        if root < 2:
            break
        mu = mobius(n)
        if mu != 0:
            s += mu / n * J_from_zeros(root, b_values)
    return s

def pi_exact_step(xs, xmax):
    xmax = int(np.ceil(xmax))
    sieve = np.ones(xmax + 1, dtype=bool)
    sieve[:2] = False
    for p in range(2, int(xmax ** 0.5) + 1):
        if sieve[p]:
            sieve[p * p::p] = False
    primes = np.nonzero(sieve)[0]
    return np.searchsorted(primes, xs, side="right")

def _equation(s2, k3):
    A = 8 * s2 ** 6 - 24 * s2 ** 4 + 24 * s2 ** 2 - 8
    return abs(A) ** (2 / 3) * (24 * k3 ** 2 * s2 ** 6 - 72 * k3 ** 2 * s2 ** 4 - 24 * k3 ** 2) + 4 * s2 **

```

```

def _get_s2(t):
    """
    Numerically solves the characteristic equation for  $k_3 = -t$  and
    returns the root  $s_2(t)$  in  $(-1, 0)$ , for any  $t \geq 1$ . This is the
    only place -- and the sole place -- where the solving happens.
    """
    return brentq(_equation, -0.9999, -0.0001, args=(-t,), xtol=1e-10)

def get_zero_imag_parts_custom(n_zeros, t_max=35, k_range=15):
    """
    Keeps the b values exactly as produced by the equation, positive AND
    negative, with no sign filter and no artificial pairing. A negative
    b coming out of the calculation is an independent solution, not the
    mirror of a positive b. Simple deduplication on the value (avoids
    near-numerical duplicates), sorted by increasing absolute value to
    favor the "closest" zeros first.
    """
    raw = []
    for t in range(1, t_max + 1):
        s2_val = _get_s2(t)
        for k in range(-k_range, k_range + 1):
            if k != 0:
                b = (math.asin(s2_val) - 2 * k * math.pi) / math.log(2)
                raw.append(b)

    raw_sorted = sorted(raw, key=abs)
    deduped = []
    for b in raw_sorted:
        if not any(abs(b - d) < 1e-5 for d in deduped):
            deduped.append(b)
        if len(deduped) >= n_zeros:
            break

    return deduped[:n_zeros]

def find_intersections(xs, exact, recon):
    """
    Detects successive intersection points between the step curve
    'exact' (integer values, discontinuous) and the continuous curve
    'recon'.

    Method: the sign of  $\text{diff} = \text{recon} - \text{exact}$  is observed between each
    pair of consecutive points on the xs grid. A sign change (or an
    exact zero value) indicates a crossing. The intersection point is
    estimated by linear interpolation of  $\text{diff}$  between the two abscissas,
    which gives a reasonable approximation of  $x_{\text{intersection}}$  even though
    'exact' is a step function (the resulting error is bounded by the
    step size of the xs grid).

    Returns a list of  $(x_{\text{intersection}}, y_{\text{intersection}})$  tuples.
    """
    intersections = []
    diff = [r - e for r, e in zip(recon, exact)]

    for i in range(1, len(xs)):
        d0, d1 = diff[i - 1], diff[i]

        if d0 == 0:
            intersections.append((float(xs[i - 1]), float(exact[i - 1])))
            continue

        if d0 * d1 < 0:
            x0, x1 = float(xs[i - 1]), float(xs[i])

```

```

        t = d0 / (d0 - d1)
        x_cross = x0 + t * (x1 - x0)
        y0, y1 = float(exact[i - 1]), float(exact[i])
        y_cross = y0 + t * (y1 - y0)
        intersections.append((x_cross, y_cross))

    return intersections

def plot_reconstruction(n_zeros, x_max, n_max, n_points, b_values, save_path):
    xs = np.linspace(2, x_max, n_points)
    exact = pi_exact_step(xs, x_max)
    recon = [pi_reconstruction(x, b_values, n_max) for x in xs]

    fig, ax = plt.subplots(figsize=(8, 5), facecolor="black")
    ax.set_facecolor("black")
    ax.step(xs, exact, color="white", where="post", label="pi(x) exact")
    ax.plot(xs, recon, color="gray", alpha=0.85, label="Reconstruction")
    ax.set_title(f"Reconstruction ({n_zeros} zeros, signed b)", color="white")
    ax.tick_params(colors="white")
    if save_path:
        fig.savefig(save_path, dpi=100, facecolor="black")

    return fig, xs, exact, recon

def main():
    # n_max = t_max * (2 * k_range) <-- positive and negative b already counted
    scenarios = [
        (100, 10, 5),
        (800, 20, 20),
        (20000, 100, 100),
    ]

    for n_zeros, t_max, k_range in scenarios:
        n_max = t_max * (2 * k_range)
        n_points = n_max

        log(f"Starting: N={n_zeros}, n_max={n_max}")
        b_values = get_zero_imag_parts_custom(n_zeros, t_max, k_range)
        save_path = os.path.join(OUT_DIR, f"recon_pm_{n_zeros}.png")

        fig, xs, exact, recon = plot_reconstruction(
            n_zeros, 80, n_max, n_points, b_values, save_path
        )
        plt.close(fig)
        gc.collect()

        if os.path.exists(save_path):
            log(f" -> Saved: {save_path}")
        else:
            log(f" -> FAILED to save {save_path}")

        # --- Detection and logging of intersections ---
        intersections = find_intersections(xs, exact, recon)
        log_intersect(f"Scenario N={n_zeros} (n_max={n_max}, x_max=80): "
            f"{len(intersections)} intersection(s) found")
        for idx, (xc, yc) in enumerate(intersections, start=1):
            log_intersect(f" N={n_zeros} | intersection #{idx}: "
                f"x = {xc:.6f}, pi(x) ~ {yc:.6f}")

if __name__ == "__main__":
    init_log()
    init_intersect_log()

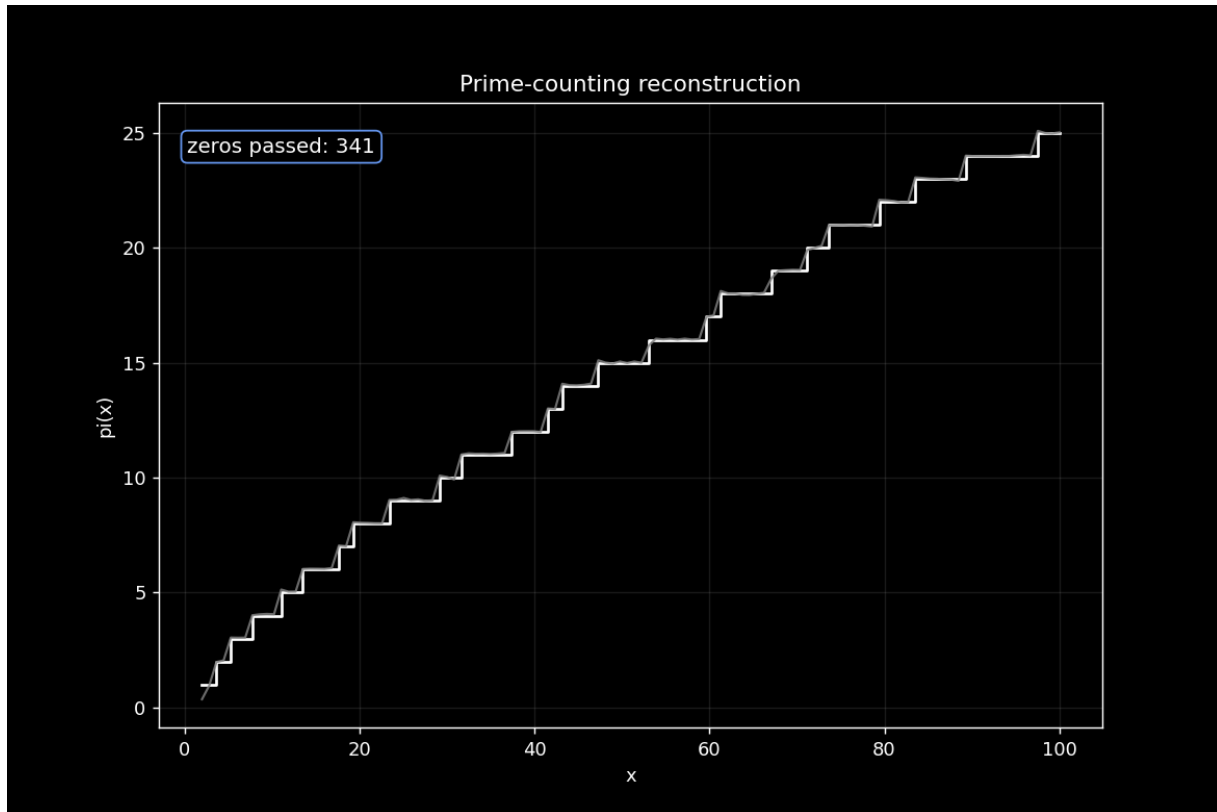
```

```

try:
    main()
except Exception:
    log(traceback.format_exc())
finally:
    if _log_file:
        _log_file.close()
    if _intersect_file:
        _intersect_file.close()

```

[Image\_2] recon\_341.png



## Algorithm 2:

```

""" Reconstruction du comptage des nombres premiers pi(x) a partir des parties imaginaires b des zeros non
triviaux de la fonction zeta. Principe (formule explicite de Riemann) : J(x) = li(x) - sum_rho li(x^rho) -
ln(2) + integrale correctrice (negligeable ici) pi(x) = sum_{n=1}^{inf} mu(n)/n * J(x^(1/n)) (inversion de
Mobius) Les zeros rho = 1/2 + i*b viennent par paires conjuguees (1/2+ib, 1/2-ib), donc chaque b > 0
contribue pour -2*Re(li(x^rho)) dans J(x). Plus on ajoute de zeros (b_1, b_2, ...), plus la courbe grise
"reconstruite" epouse la marche d'escalier blanche de pi(x) exact. CORRECTIF (bug de phase) : -----
----- La version precedente calculait li(x^rho) en passant par z = x**rho (mpmath), qui repasse
ensuite par log(z) en interne. Or log() ramene toujours la partie imaginaire dans (-pi, pi] : pour un b
eleve, rho*log(x) peut valoir plusieurs centaines (ex. 308.9 pour b~800 et x=90), et ce "nombre de tours"
est perdu des qu'on repasse par x**rho puis log(...). Le resultat redevient une valeur quelconque dans (-
pi, pi], ce qui fait diverger complètement la reconstruction des qu'on ajoute des zeros a b eleve (courbe
qui plonge vers -250 au lieu de suivre pi(x) exact). Correction : on calcule directement Ei(rho * log(x))
via mpmath.ei, sans jamais repasser par x**rho ni par un second log(). C'est a la fois plus rapide (une
seule evaluation de log par x) et mathematiquement correct, puisque li(x^rho) = Ei(log(x^rho)) = Ei(rho *
log(x)) par definition, sans ambiguïte de branche a lever ici. MODIFICATION : les images sont enregistrees
dans le meme repertoire de sortie que l'algorithm2 (OUT_DIR ci-dessous), pour faciliter la comparaison
des courbes des deux scripts. """

```



```

import os
import numpy as np
import matplotlib.pyplot as plt
from mpmath import mp, mpf, mpc, li, log, ei, zetazero

mp.dps = 25 # precision decimale

# -----
# 0) Repertoire de sortie commun avec algorithme_2
# -----
BASE_DIR = "/storage/emulated/0/A_algorithms-Python"
OUT_DIR = os.path.join(BASE_DIR, "outputs3")
os.makedirs(OUT_DIR, exist_ok=True)

# -----
# 1) Fonction de Mobius (pour l'inversion)
# -----
def mobius(n):
    if n == 1:
        return 1
    result = 1
    n0 = n
    p = 2
    while p * p <= n0:
        if n0 % p == 0:
            n0 //= p
            if n0 % p == 0:
                return 0
            result = -result
        p += 1
    if n0 > 1:
        result = -result
    return result

# -----
# 2) J(x) tronque a N zeros (b_values = liste des parties imaginaires)
# CORRIGE : Ei(rho*log(x)) direct, plus de x**rho / log() intermediaire
# -----
def J_from_zeros(x, b_values):
    x = mpf(x)
    if x <= 1:
        return mpf(0)
    log_x = log(x) # calcule une seule fois, reutilise pour tous les zeros
    total = li(x) # li(x) reel : aucun probleme de branche ici
    for b in b_values:
        rho = mpc(0.5, b)
        total -= 2 * ei(rho * log_x).real # <-- correction du bug de phase
    total -= log(2)
    return total

# -----
# 3) pi(x) reconstruit par inversion de Mobius de J
# -----
def pi_reconstruction(x, b_values, n_max=25):
    x = mpf(x)
    s = mpf(0)
    n = 1
    while n <= n_max:
        root = x ** (mpf(1) / n)
        if root < 2:
            break
        mu = mobius(n)
        if mu != 0:
            s += mpf(mu) / n * J_from_zeros(root, b_values)
        n += 1

```

```

return s

# -----
# 4) pi(x) exact (marche d'escalier), via crible simple
# -----
def pi_exact_step(xs, xmax):
    xmax = int(np.ceil(xmax))
    sieve = np.ones(xmax + 1, dtype=bool)
    sieve[2] = False
    for p in range(2, int(xmax ** 0.5) + 1):
        if sieve[p]:
            sieve[p * p::p] = False
    primes = np.nonzero(sieve)[0]
    return np.searchsorted(primes, xs, side="right")

# -----
# 5) Recuperation des N premiers zeros de zeta (calculés une seule fois)
# -----
def get_zero_imag_parts(n_zeros):
    return [float(zetazero(k).imag) for k in range(1, n_zeros + 1)]

# -----
# 6) Trace pour un nombre de zeros donne
# -----
def plot_reconstruction(n_zeros, x_max=100, n_points=150, b_values=None, save_path=None):
    if b_values is None:
        b_values = get_zero_imag_parts(n_zeros)
    else:
        b_values = b_values[:n_zeros]

    xs = np.linspace(2, x_max, n_points)
    exact = pi_exact_step(xs, x_max)
    recon = [float(pi_reconstruction(x, b_values)) for x in xs]

    fig, ax = plt.subplots(figsize=(9, 6), facecolor="black")
    ax.set_facecolor("black")
    ax.step(xs, exact, color="white", where="post", linewidth=1.5, label="pi(x) exact")
    ax.plot(xs, recon, color="gray", linewidth=1.2, alpha=0.85, label="Reconstruction")

    ax.set_title("Prime-counting reconstruction", color="white")
    ax.set_xlabel("x", color="white")
    ax.set_ylabel("pi(x)", color="white")
    ax.tick_params(colors="white")
    for spine in ax.spines.values():
        spine.set_color("white")
    ax.grid(alpha=0.15)

    ax.text(0.03, 0.92, f"zeros passed: {n_zeros}", transform=ax.transAxes,
           color="white", fontsize=11,
           bbox=dict(boxstyle="round", facecolor="black", edgecolor="cornflowerblue"))

    if save_path:
        fig.savefig(save_path, dpi=130, facecolor="black")
    return fig, ax

if __name__ == "__main__":
    # Exemple : calcule une fois les N premiers zeros, puis trace
    # plusieurs etapes de reconstruction (comme dans l'animation de reference).
    N_MAX = 79 # on couvre bien la plus grande etape demandee (79)
    print("Calcul des zeros de zeta (peut prendre un moment)...")
    b_values = get_zero_imag_parts(N_MAX)
    print("Premiers b (parties imaginaires) :", [round(b, 4) for b in b_values[:5]], "...")

    for n in [16, 46, 79]:
        save_path = os.path.join(OUT_DIR, f"recon_{n}.png")

```

```

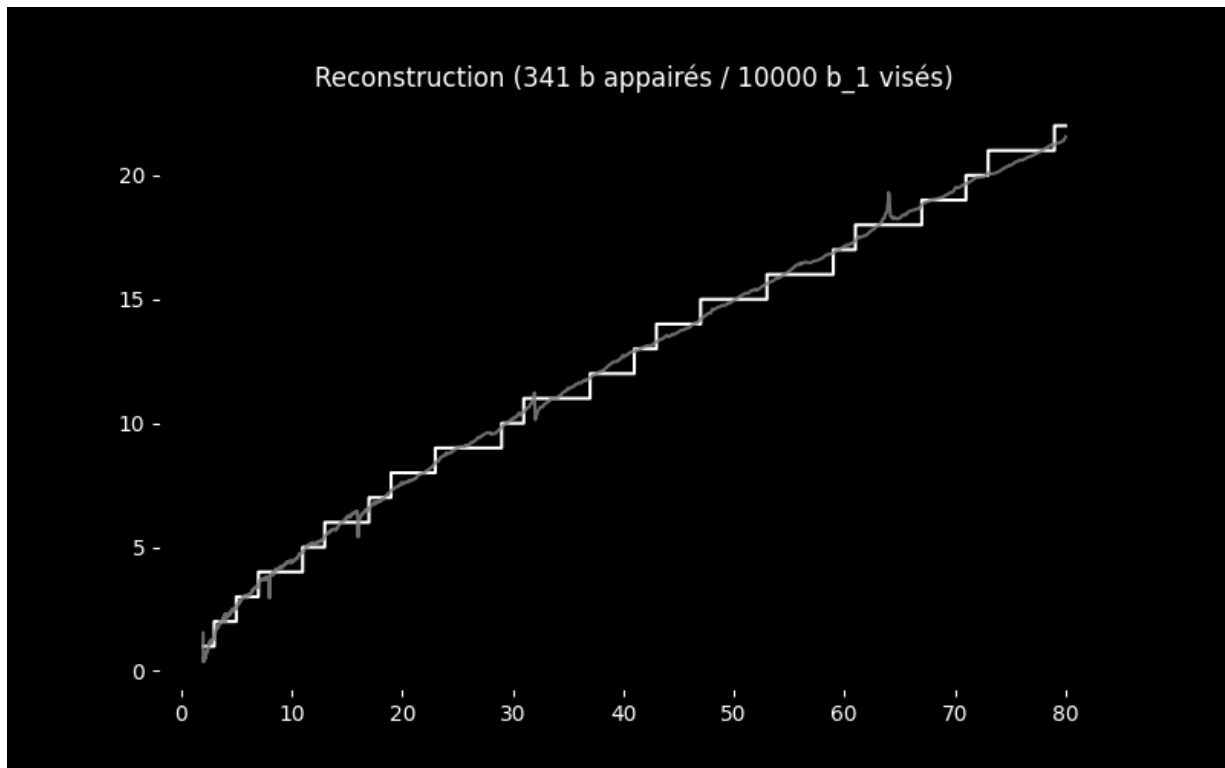
print(f"Generation du graphe pour {n} zeros...")
fig, ax = plot_reconstruction(n, x_max=100, n_points=120,
                             b_values=b_values,
                             save_path=save_path)

plt.close(fig)
print(f"Image enregistrée : {save_path}")

print("Images générées dans :", OUT_DIR)

```

**[Image\_3]** The introduction of truly random "b" values radically alters the steplike appearance of the curve: gone are the steps with variable slopes or those aiming for precision; we believe that as  $n$  approaches infinity, the *\*b<sub>reference</sub>\** curves—given that approximately 67.25% of possible {b} values are involved—degenerate or come to resemble this final curve. (recon\_match\_10000 (initial).png, recon\_341.png)



## Conclusion

This debate can be considered to reach a major conceptual resolution in this body of work: the divergent-formula approach ( $\rightarrow$  *b<sub>mathematical</sub>*) does not seek to match the trajectory of individual known zeros step by step, but instead demonstrates that, by working at the level of the global resonant subsystem, it makes it possible to surpass the 67.2% barrier and move toward 100% spectral yield, while offering a direct mechanism for identifying primality.